

YAZILIM GELİŞTİRME METODOLOJİSİ VE YAŞAM DÖNGÜSÜ

METODOLOJILER



- **Metodoloji:** Bir BT projesi ya da yazılım yaşam döngüsü aşamaları boyunca kullanılacak ve birbirleriyle uyumlu yöntemler bütünü.
- Bir metodoloji,
 - bir süreç modelini ve
 - belirli sayıda belirtim yöntemini içerir
- Günümüzdeki metodolojiler genelde **Çağlayan** ya da **Helezonik model**i temel almaktadır

BİR METODOLOJIDE BULUNMASI GEREKEN TEMEL BİLEŞENLER (ÖZELLİKLER)

- Ayrıntılandırılmış bir süreç modeli
- Ayrıntılı süreç tanımları
- İyi tanımlı üretim yöntemleri
- Süreçlerarası arayüz tanımları
- Ayrıntılı girdi tanımları
- Ayrıntılı çıktı tanımları
- Proje yönetim modeli
- Konfigürasyon yönetim modeli
- Maliyet yönetim modeli
- Kalite yönetim modeli
- Risk yönetim modeli
- Değişiklik yönetim modeli
- Kullanıcı arayüz ve ilişki modeli
- Standartlar

BİR METODOLOJIDE BULUNMASI GEREKEN

TEMEL BİLEŞENLER

- Metodoloji bileşenleri ile ilgili olarak bağımsız kuruluş (IEEE, ISO, vs.) ve kişiler tarafından geliştirilmiş çeşitli standartlar ve rehberler mevcuttur.
- Kullanılan süreç modelleri ve belirtim yöntemleri zaman içinde değiştiği için standart ve rehberler de sürekli güncellenmektedir.
- Bir kuruluşun kendi metodolojisini geliştirmesi oldukça kapsamlı, zaman alıcı ve uzmanlık gerektiren bir faaliyet olup, istatistikler yaklaşık 50 kişi/ay'lık bir iş gücü gerektirdiğini göstermektedir.





YAZILIM GELİŞTİRME YAŞAM DÖNGÜSÜ

- Yazılımın hem üretim, hem de kullanım süreci boyunca geçirdiği tüm aşamalar **yazılım geliştirme yaşam döngüsü** olarak tanımlanır.
- Yazılım işlevleri ile ilgili gereksinimler sürekli olarak değiştiği ve genişlediği için, söz konusu aşamalar sürekli bir döngü biçiminde ele alınır.
- Döngü içerisinde her hangi bir aşamada geriye dönmek ve tekrar ilerlemek söz konusudur.
- Yazılım yaşam döngüsü tek yönlü ve doğrusal değildir.

YAZILIM YAŞAM DÖNGÜSÜ TEMEL ADIMLARI

1. Analiz

Sistem gereksinimlerinin ve işlevlerinin ayrıntılı olarak çıkarıldığı aşama. Var olan işler incelenir, temel sorunlar ortaya çıkarılır.

2. Planlama

Personel ve donanım gereksinimlerinin çıkarıldığı, fizibilite çalışmasının yapıldığı ve proje planının oluşturulduğu aşamadır.

3. Tasarım

Belirlenen gereksinimlere yanıt verecek yazılım sisteminin temel yapısının oluşturulduğu aşamadır.

mantıksal; önerilen sistemin yapısı anlatılır,

fiziksel; yazılımı içeren bileşenler ve bunların ayrıntıları.

4. Gerçekleştirim

Kodlama, test etme ve kurulum çalışmalarının yapıldığı aşamadır.

5. Bakım

Hata giderme ve yeni eklentiler yapma aşaması (teslimden sonra).

YAZILIM YAŞAM DÖNGÜSÜ TEMEL ADIMLARI

- Yaşam döngüsünün temel adımları **çekirdek süreçler (core processes)** olarak da adlandırılır.
- Bu süreçlerin gerçekleştirilmesi amacıyla
 - **Belirtim (specification) yöntemleri** - bir çekirdek sürece ilişkin fonksiyonları yerine getirmek amacıyla kullanılan yöntemler
 - **Süreç (process) modelleri** - yazılım yaşam döngüsünde belirtilen süreçlerin geliştirme aşamasında, hangi düzen ya da sırada, nasıl uygulanacağını tanımlayan modeller kullanılır.

BELİRTİM YÖNTEMLERİ

○ Süreç Akışı İçin Kullanılan Belirtim Yöntemleri

Süreçler arası ilişkilerin ve iletişimin gösterildiği yöntemler
(Veri Akış Şemaları, Yapısal Şemalar, Nesne/Sınıf Şemaları).

○ Süreç Tanımlama Yöntemleri

Süreçlerin iç işleyişini göstermek için kullanılan yöntemler
(Düz Metin, Algoritma, Karar Tabloları, Karar Ağaçları, Anlatım Dili).

○ Veri Tanımlama Yöntemleri

Süreçler tarafından kullanılan verilerin tanımlanması için kullanılan yöntemler.

(Nesne İlişki Modeli, Veri Tabanı Tabloları, Veri Sözlüğü).

YAZILIM SÜRECİ MODELLERİ

- Yazılım üretim işinin genel yapılma düzenine ilişkin rehberlerdir.
 - Süreçlere ilişkin ayrıntılarla ya da süreçler arası ilişkilerle ilgilenmezler.
1. Gelişigüzel Model
 2. Barok Modeli
 3. Çağlayan (Şelale) Modeli
 4. V Modeli
 5. Helezonik (Spiral) Model
 6. Evrimsel Model
 7. Artırımsal Model
 8. Araştırma Tabanlı Model

GELİŞİGÜZEL MODEL

- Herhangi bir model ya da yöntem yok.
- Geliştiren kişiye bağımlı (belli bir süre sonra o kişi bile sistemi anlayamaz ve geliştirme güçlüğü yaşar).
- İzlenebilirliği ve bakımı oldukça zor.
- 60'lı yıllarda.
- Genellikle tek kişilik üretim ortamı.
- Basit programlama.

BAROK MODELİ

- İnceleme
- Analiz
- Tasarım
- Kodlama
- Modül Testleri
- Alt sistem Testleri
- Sistem Testi
- Belgeleme
- Kurulum

Yaşam döngüsü temel adımlarının doğrusal bir şekilde geliştirildiği model.

70'li yıllar.

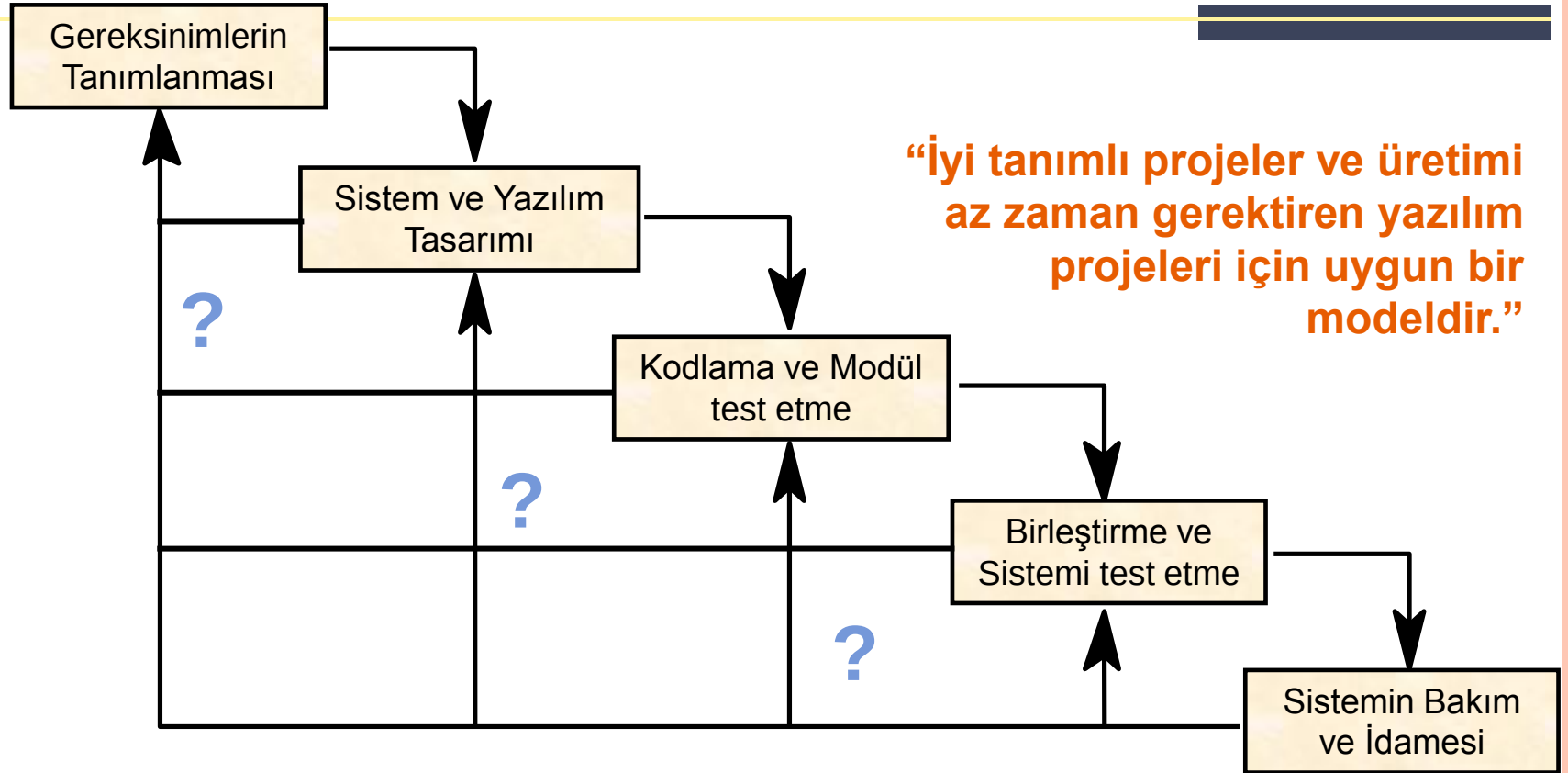
Belgelemeyi ayrı bir süreç olarak ele alır, ve yazılımın geliştirilmesi ve testinden hemen sonra yapılmasının öngörür.

Halbuki, günümüzde belgeleme yapılan işin doğal bir ürünü olarak görülmektedir.

Aşamalar arası geri dönüşlerin nasıl yapılacağı tanımlı değil.

“ Gerçekleştirim aşamasına daha fazla ağırlık veren bir model olup, günümüzde kullanımı önerilmemektedir. ”

ÇAĞLAYAN (ŞELELE) MODELİ



“Geleneksel model olarak da bilinen bu modelin kullanımı günümüzde giderek azalmaktadır.”

“Yazılım yaşam döngüsü adımları baştan sona en az bir kez izlenir.”

ÇAĞLAYAN (ŞELEALE) MODELİ

- Barok modelin aksine **belgeleme** işlevini ayrı bir aşama olarak ele almaz ve üretimin doğal bir parçası olarak görür.
- Barok modele göre geri dönüşler iyi tanımlanmıştır.
- Yazılım tanımlamada belirsizlik yok (ya da az) ise ve yazılım üretimi çok zaman almayacak ise uygun bir süreç modelidir.

ÇAĞLAYAN (ŞELELE) MODELİ

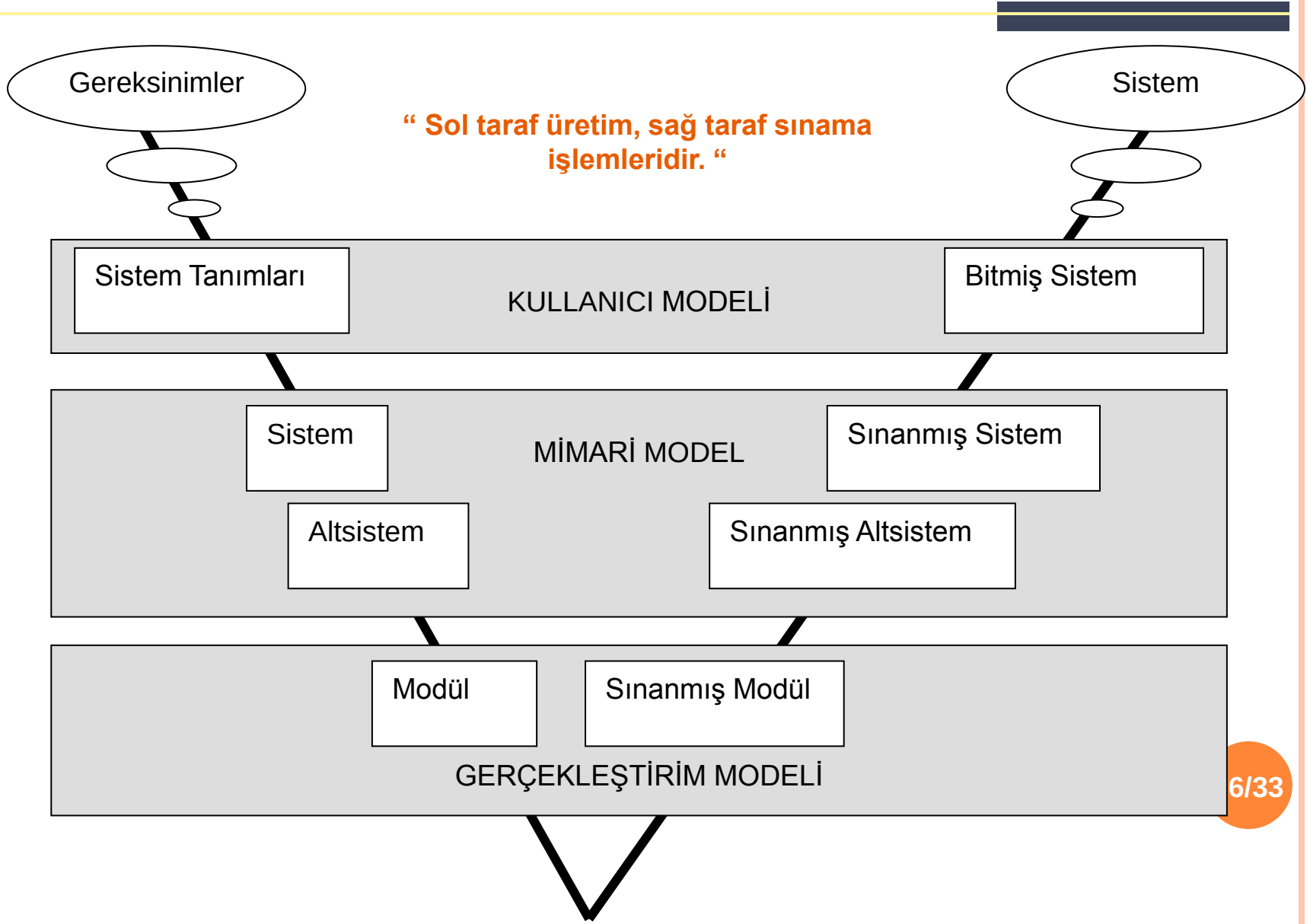
SORUNLARI



- Gerçek yaşamdaki projeler genelde yineleme gerektirir.
- Genelde yazılımın kullanıcıya ulaşma zamanı uzundur.
- Gereksinim tanımlamaları çoğu kez net bir şekilde yapılamadığından dolayı, yanlışların düzeltilme ve eksiklerin giderilme maliyetleri yüksektir.
- Yazılım üretim ekipleri bir an önce program yazma, çalıştırma ve sonucu görme eğiliminde olduklarından, bu model ile yapılan üretimlerde ekip mutsuzlaşmakta ve kod yazma dışında kalan (ve iş yükünün %80'ini içeren) kesime önem vermemektedirler.
- Üst düzey yönetimlerin ürünü görme süresinin uzun oluşu, projenin bitmeyeceği ve sürekli gider merkezi haline geldiği düşüncesini yaygınlaştırmaktadır.

V SÜREÇ MODELİ

“Belirsizliklerin az iş tanımlarının belirgin olduğu bilişim teknolojileri projeleri için uygun bir modeldir. “



V SÜREÇ MODELİ

V süreç modelinin temel çıktıları;

- **Kullanıcı Modeli**

Geliştirme sürecinin kullanıcı ile olan ilişkileri tanımlanmakta ve sistemin nasıl kabul edileceğine ilişkin sınaama belirtileri ve planları ortaya çıkarılmaktadır.

- **Mimari Model**

Sistem tasarımı ve oluşacak alt sistem ile tüm sistemin sınaama işlemlerine ilişkin işlevler.

- **Gerçekleştirim Modeli**

Yazılım modüllerinin kodlanması ve sınaanmasına ilişkin fonksiyonlar.

Model, kullanıcının projeye katkısını arttırmaktadır.

HELEZONİK(SPIRAL) MODELİ

Planlama

Amaca, Alternatiflere ve Sınırlamalara karar verme

Risk Analizi

Alternatifleri değerlendirme ve risk analizi

Yazılım Mühendisliği

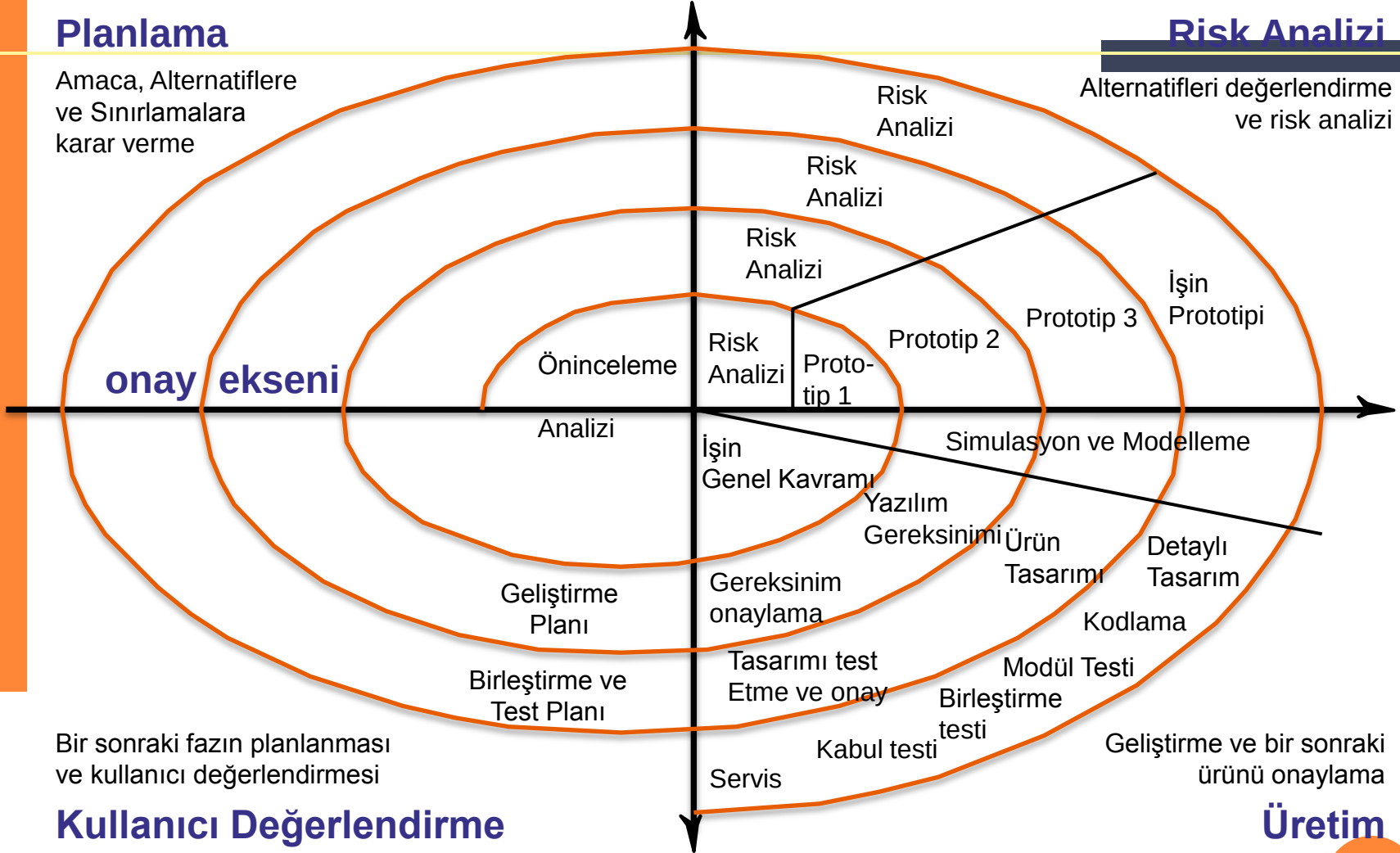
onay eksenini

Bir sonraki fazın planlanması ve kullanıcı değerlendirmesi

Kullanıcı Değerlendirme

Üretim

18/33



HELEZONIK MODEL

1. Planlama

Üretilcek ara ürün için planlama, amaç belirleme, bir önceki adımda üretilen ara ürün ile bütünleştirme

2. Risk Analizi

Risk seçeneklerinin araştırılması ve risklerin belirlenmesi

3. Üretim

Ara ürünün üretilmesi

4. Kullanıcı Değerlendirmesi

Ara ürün ile ilgili olarak kullanıcı tarafından yapılan sınama ve değerlendirmeler

HELEZONİK MODELİN AVANTAJLARI

1. Kullanıcı Katkısı

Üretim süreci boyunca ara ürün üretme ve üretilen ara ürünün kullanıcı tarafından sınanması temeline dayanır.

Yazılımı kullanacak personelin sürece erken katılması ileride oluşabilecek istenmeyen durumları engeller.

2. Yönetici Bakışı

Gerek proje sahibi, gerekse yüklenici tarafındaki yöneticiler, çalışan yazılımlarla proje boyunca karşılaştıkları için daha kolay izleme ve hak ediş planlaması yapılır.

3. Yazılım Geliştirici (Mühendis) Bakışı

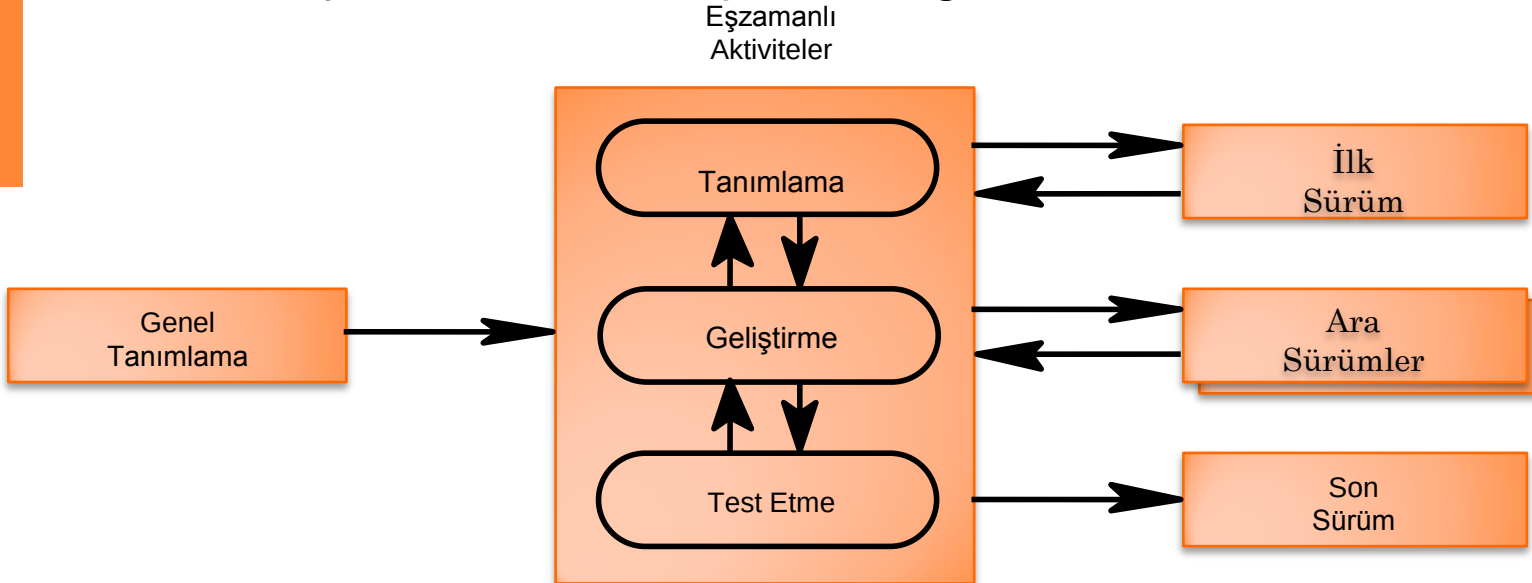
Yazılımın kodlanması ve sınanması daha erken başlar.

HELEZONIK MODEL

- Risk Analizi Olgusu ön plana çıkmıştır.
- Her döngü bir fazı ifade eder. Doğrudan tanımlama, tasarım,... vs gibi bir faz yoktur.
- Yinelemeli artımsal bir yaklaşım vardır.
- Prototip yaklaşımı vardır.

EVİRİMSEL GELİŞTİRME SÜREÇ MODELİ

- İlk tam ölçekli modeldir.
- Coğrafik olarak geniş alana yayılmış, çok birimli organizasyonlar için önerilmektedir (**banka uygulamaları**).
- Her aşamada üretilen ürünler, üretildikleri alan için tam işlevselliği içermektedirler.
- **Pilot uygulama** kullan, test et, güncelle diğer birimlere taşı.
- Modelin başarısı **ilk evrimin başarısına** bağlıdır.



Evrimsel Geliştirme Süreç Modeli

- Banka ve şubeleri olan işletme uygulamaları.
- Önce sistem geliştirilir ve adım adım dağıtılır.
- Her adımda geri besleme ile düzeltmeler yapılır.

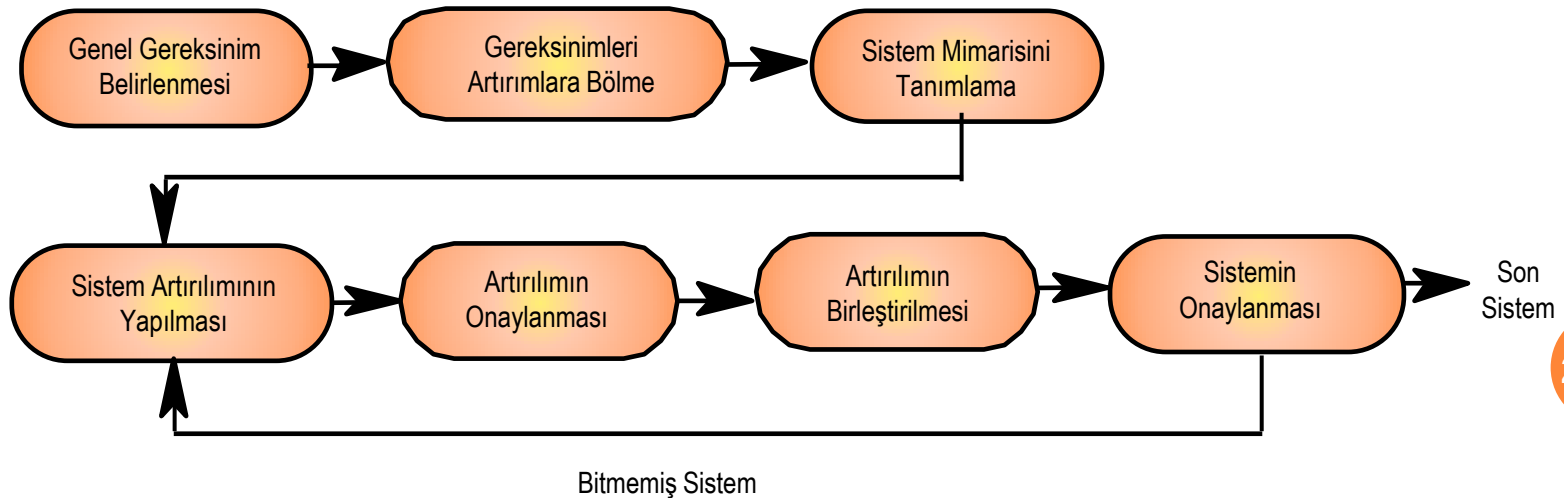
Sorunlar :

- Değişiklik denetimi
- Konfigürasyon Yönetimidir
 - Sürüm Yönetimi
 - Değişiklik Yönetimi
 - Kalite Yönetimi



ARTIRIMSAL GELİŞTİRME SÜREÇ MODELİ

- Üretilen **her yazılım sürümü birbirini kapsayacak** ve giderek artan sayıda işlev içerecek şekilde geliştirilir.
- Öğrencilerin bir dönem boyunca geliştirmeleri gereken bir programlama ödevinin 2 haftada bir gelişiminin izlenmesi (bitirme tezleri).
- Uzun zaman alabilecek ve sistemin eksik işlevlikle çalışabileceği türdeki projeler bu modele uygun olabilir.
- Bir taraftan kullanım, diğer taraftan üretim yapılır.



ARAŞTIRMA TABANLI SÜREÇ MODELİ

- Araştırma ortamları **bütünüyle belirsizlik** üzerine çalışan ortamlardır.
- Yap-at **prototipi** olarak ta bilinir.
- Yapılan işlerden edinilecek sonuçlar belirgin değildir.
- Geliştirilen **yazılımlar genellikle sınırlı sayıda kullanılır** ve kullanım bittikten sonra işe yaramaz hale gelir ve atılır.
- Model-zaman-fiyat kestirimi olmadığı için **sabit fiyat sözleşmelerinde uygun değildir.**



- En Hızlı Çalışan asal sayı test programı!
- En Büyük asal sayıyı bulma programı!
- Satranç programı!
- ...

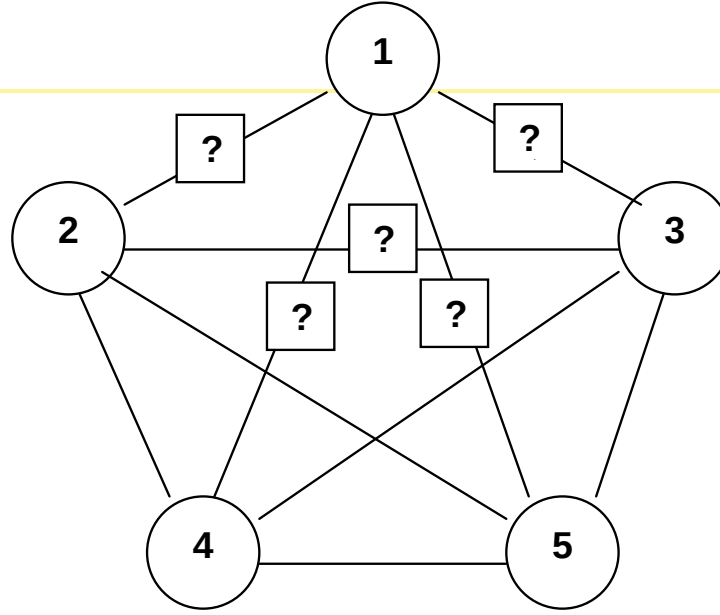
YOURDON YAPISAL SİSTEM TASARIM METODOLOJISI

Kolay uygulanabilir bir model olup, günümüzde oldukça yaygın olarak kullanılmaktadır.

Çağlayan modelini temel almaktadır.

Bir çok CASE aracı tarafından doğrudan desteklenmektedir.

Aşama	Kullanılan Yöntem ve Araçlar	Ne için Kullanıldığı	Çıktı
Planlama	Veri Akış Şemaları, Süreç Belirtilimleri, Görüşme, Maliyet Kestirim Yöntemi, Proje Yönetim Araçları	Süreç İnceleme Kaynak Kestirimi Proje Yönetimi	Proje Planı
Analiz	Veri Akış Şemaları, Süreç Belirtilimleri, Görüşme, Nesne ilişki şemaları Veri	Süreç Analizi Veri Analizi	Sistem Analiz Raporu
Analizden Tasarıma Geçiş	Akışa Dayalı Analiz, Süreç belirtilimlerinin program tasarım diline dönüştürülmesi, Nesne ilişki şemalarının veri tablosuna dönüştürülmesi	Başlangıç Tasarımı Ayrıntılı Tasarım Başlangıç Veri tasarımı	Başlangıç Tasarım Raporu
Tasarım	Yapısal Şemalar, Program Tasarım Dili, Veri Tabanı Tabloları	Genel Tasarım Ayrıntılı Tasarım Veri Tasarımı	Sistem Tasarım Raporu



- 1: Düşük Maliyetli Gerçekleştirim**
- 2: Bakım Kolaylığı**
- 3: Zamanında Teslim**
- 4: Güvenirliği Yüksek Yazılım**
- 5: Uzmanca Yazılım Geliştirme**

Uygulama Yazılımı Geliştirmede Çelişen Tercihler



Pilot uygulama yapmaya üç nedenle başvurulur:

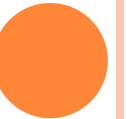


Kaynak :

www.mehmetduran.com

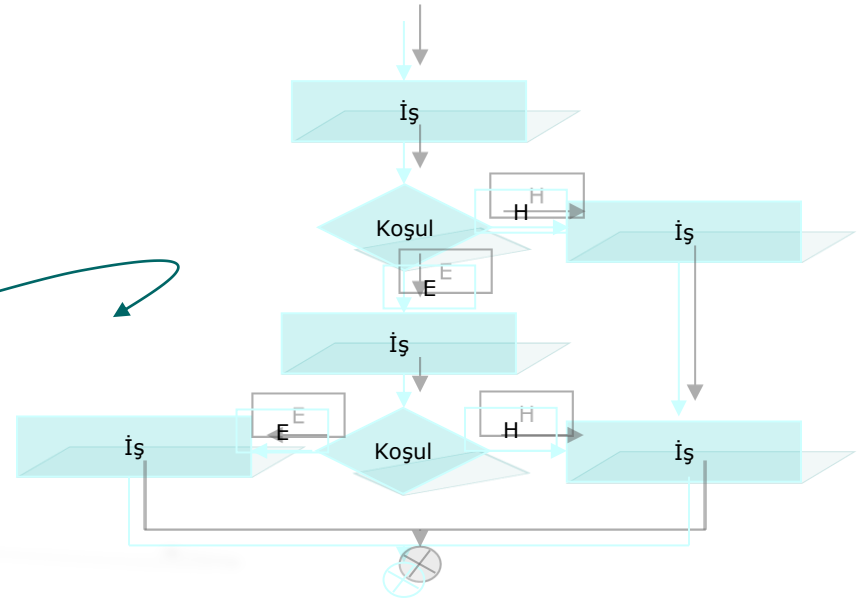
03/2007 - Yrd.Doç.Dr. G.YILMAZ Yazılım Geliştirme Yaşam Döngüsü

ÖRNEK ÇALIŞMA



Sistem

Tasarımı



Tasarım , herhangi bir mühendislik sürecindeki ilk adımdır.

Genel olarak deneyim bilgi birikimiyle desteklenen çeşitli kurallarla yapılır.

Çeşitli geliştirme teknikleri , tanımlama ve tasarım yöntemleri bulunsa da Yazılım mühendisliği hala bir “**sanat**” niteliğindedir.

En önemli adımlarından birisi **Veri Tasarımı** dır.

Çözümleme sırasında toplanan bilgilerin kullanılacak veri yapılarına dönüştürülmesini içerir.

Mimari Tasarım ;

Yazılım birimlerinin yapısal parçalarını , birbirleriyle ilişkilerini tanımlar.

Yordamsal Tasarım ;

Yazılılımı oluşturan yapısal birimler yordam ve fonksiyonlar haline dönüştülür.

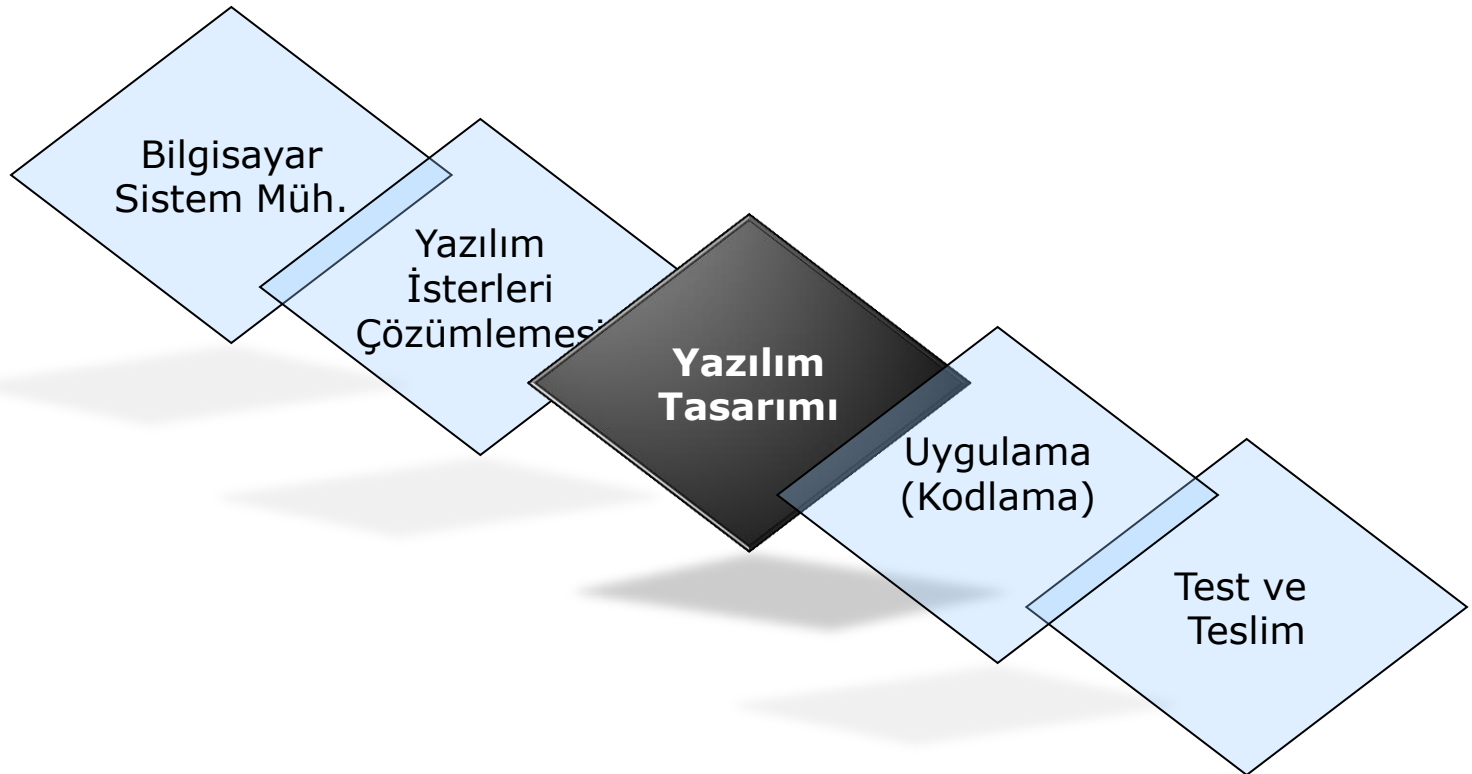
Arayüz Tasarımı ;

İnsan – Makine etkileşiminin şeklini alt sistemlerle olan arayüzlerin ayrıntılarını içerir.

Tüm detaylar belgede toplanır , değerlendirilir sonra da kodlama aşamasına geçilir.

Tasarım , yazılım testine kadar her şeyi etkilediğinden nitelik unsurunun öne çıktığı ilk aşama özelliğini taşımaktadır.

Yazılım geliştirme süreci içerisinde tasarım aşamasının yeri



Tasarımın ilk amacı Basitlik Olmalıdır.

“Sistem öyle tasarlanmalıdır ki, bir dizi değişiklik yapılsa bile sistem tasarımı hala basit kalabilmelidir.”

Yazılımın tasarımı gözden geçirilmesi gereken temel ilkelerden en önemlileri şunlardır ;

1. **Soyutlama** : Denetimi ve anlaşılabilirliği arttırmak üzere en az ayrıntı ve işlem yapmaktır.
2. **Bilgi Gizleme** : Modüllerin iç yapılarını diğerlerinden gizlemek, bu şekilde karmaşıklığı engellemek ve soyutlamayı arttırmaktır.
3. **Kapsama** : Tüm isterlerin eksiksiz olarak karşılanması amacıyla yordam ve verilerin denetim altına alınması.

Tasarım Nitelikleri :

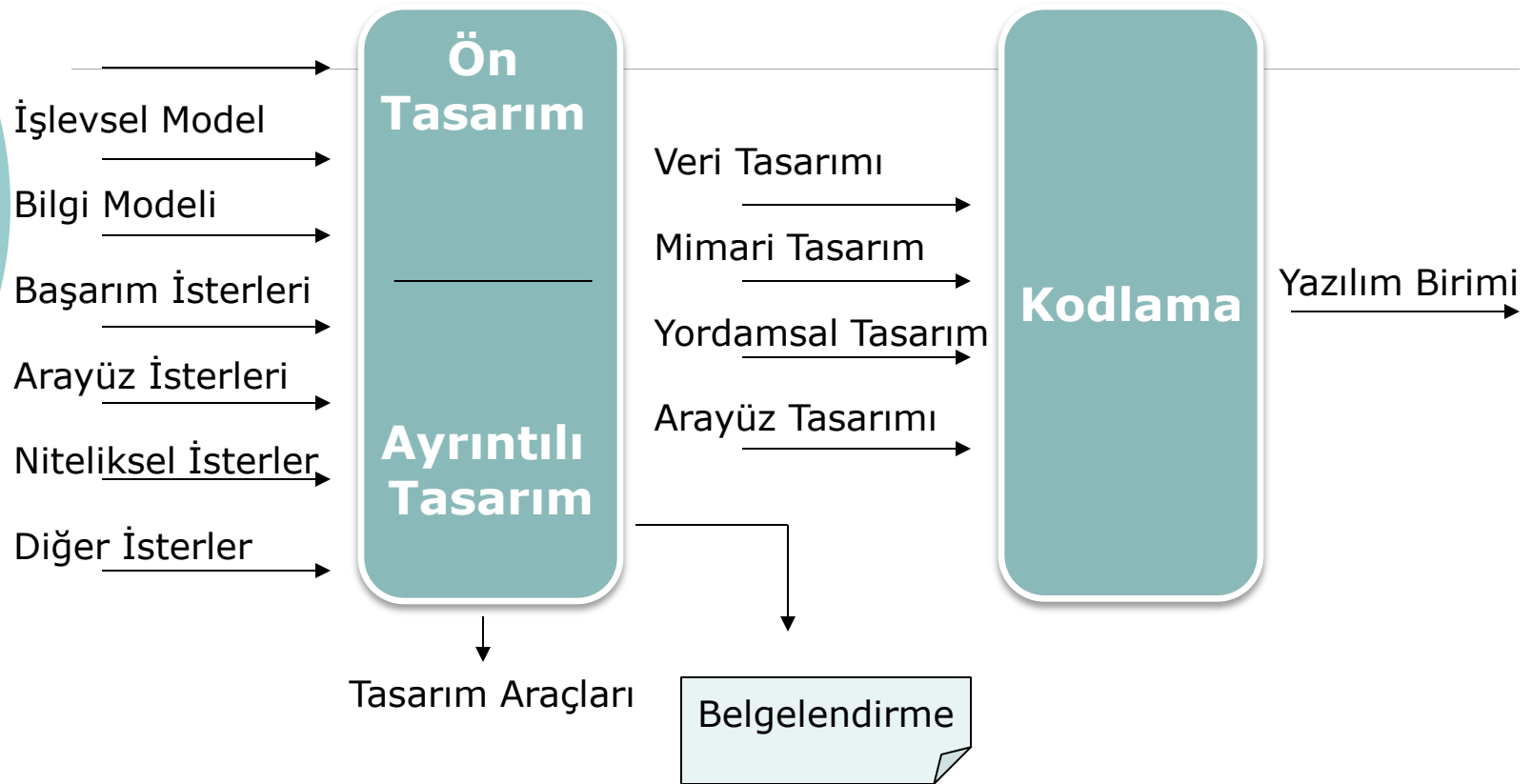
1. İsterler ile izlenebilirliği olmalıdır.
2. Geliştirilen birimin kodu ve testleri ile izlenebilirliği olmalıdır.
3. Programlama dilinden mümkün olduğunca bağımsız olmalıdır.
4. İşlevselliği , başarımı ve güvenilirliği yüksek bir ürün oluşturulmalıdır.
5. Yürütme sırasında oluşabilecek hataların ilgili iş sürecini aksatmayacak şekilde kotarılması sağlanmalıdır.
6. Öğrenmesi ve kullanımı kolay bir ürünü hedeflemelidir.
7. Tekrar kullanılabilir olmalıdır.
8. Bir ürün ailesine temel oluşturabilmelidir.
9. Kolay anlaşılmalıdır.
10. Gerektiğinde kolaylıkla değiştirilebilmelidir.
11. Kurumsal tasarım standartlarına uygun olmalıdır.
12. Diğer tasarımlarla birleştirilebilmesi mümkün olmalıdır.

Yazılım tasarımı sürecinde ve tanımlamalarında rehber olarak bazı standartlar kullanılabilir.

Yönetsel olarak süreç iki aşamada ele alınabilir :

- **Ön tasarım** : isterlerin veri ve mimari tasarımına dönüştürülmesidir.
- **Ayrıntılı tasarım** : Veri ve mimari tasarımın ayrıntılı veri yapıları ile algoritmik gösterime dönüştürülmesidir.

Yazılım geliştirme sürecinin tasarım aşaması sırasında kullanılan veri akışı :



Veri Tasarımı :

Veri yapısı ve veri modeli iç içe geçmiş iki kavramdır.

Birisi verinin bellekte tutulması veya saklanmasıyla ilgilenirken diğeri veriler arasındaki ilişki ve bağıntılar konusıyla ilgilenir.

Veriler üzerinde işlem yapacak olan algoritmalar da bu veri modellerine göre tasarlanırlar.

İyi bir veri tasarımı için neler gereklidir ;

- Veri yapıları/veri modelleri üzerinde yapılacak işlemlerin tanımlanması.
- Veri sözlüğünün oluşturulması.
- Döngüsel bir yol izlenmesi.
- Veri yapıları yalnızca kendilerini kullanan modüllere görünür olmalıdır.
- Sık kullanılan veri modelleri kütüphane haline getirilmelidir.
- Programlama dili özellikleri kullanılarak kodlama yapılmalıdır.

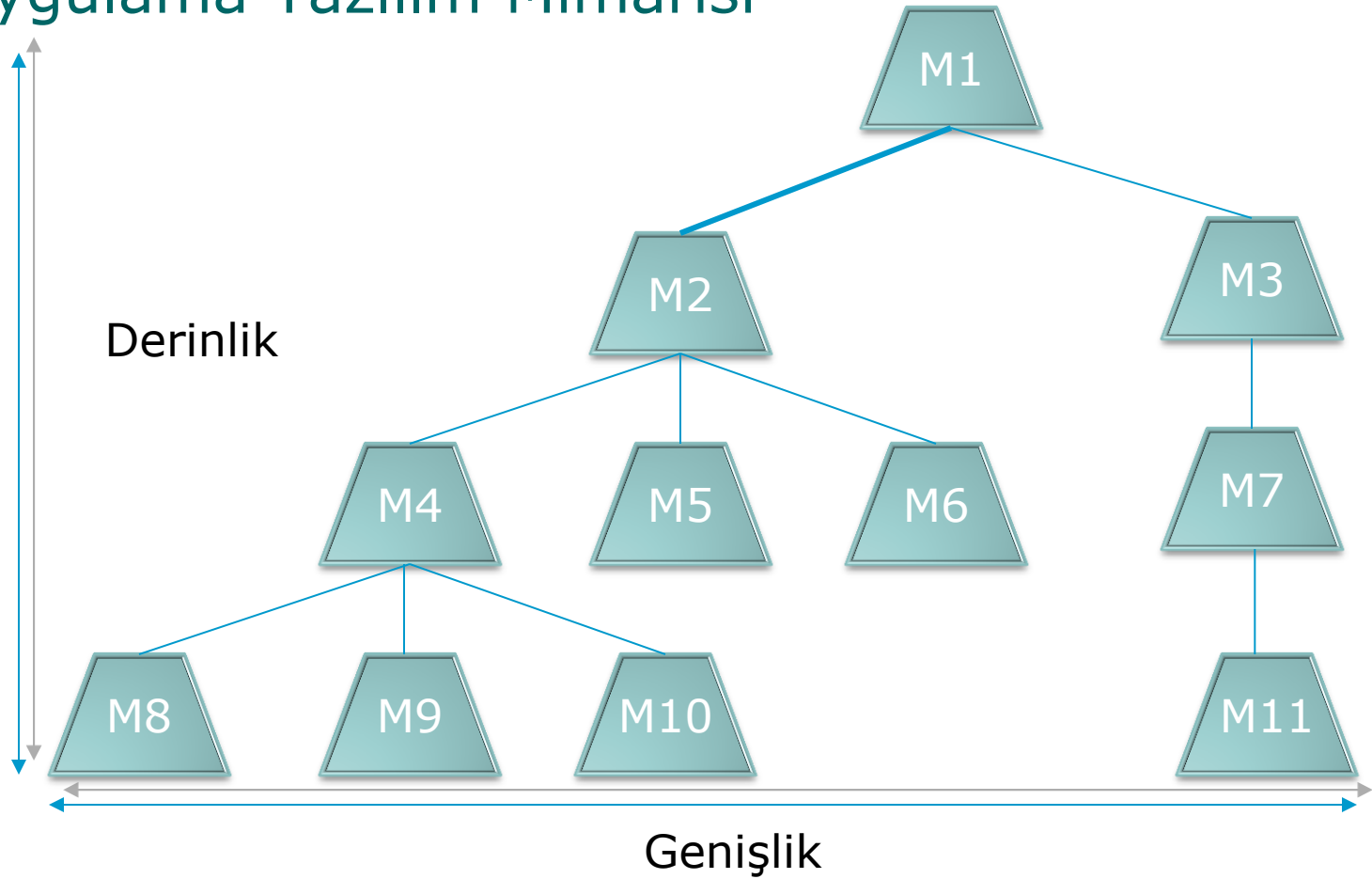
Mimari Tasarım ;

Uygulama yazılımı bir problemin çözümünü çeşitli parçalara bölerek sağlayabilir.

Parçaların yazılımdaki karşılığı **modüller** dir

Modüllerin sıradüzensel ilişkilerini gösteren yapıya **uygulama yazılım mimarisi** denir.

Uygulama Yazılım Mimarisi



- Uygulama alanının özellikleri :
Donanım özellikleri ...
- Uygulama yazılımının karmaşıklık derecesi :
Basit uygulamalar , tek program içinde , hertürlü arayüz ve bilgi işlemeyi kapsayacak şekilde geliştirilebilirler.Bölümlemek.
- Kullanıcı arayüzü kısıtlamaları :
Bilgi işleme birimleri ile kullanıcı arayüzünün farklı mimariye sahip işlemcilerde çalışması gereken durumlar olabilir.
- Taşınabilirlik :
Farklı işletim sistemi ve donanım özelliklerinde de çalışabilmesi gereklidir.

Yapısal Programlama Gösterimi :

Yazılım tarihinin en eski tasarım yöntemlerinden biri belirli yapıları kullanarak işlevleri metinsel bir şekilde anlatmaktır.

Tasarım dillerinin ortak özellikleri :

- Her türlü yapıyı destekleyebilen sabit bir anahtar sözcük listesi.
- Veri tipleri ve veri yapıları tanımlama yeteneği.
- Alt program tanımlama ve çağırma düzeneği.
- Bilgi işlemeyi serbest bir dille anlatabilme olanağı.
- Arayüz tanımlama yeteneği.
- Koşul ve çevrim yapıları.
- Giriş / Çıkış yapıları
- Zaman belirtimleri.

Grafiksel Gösterim :

Bazen bir resim bir çok satırdan oluşan bir anlatım yerine geçebilir.

Bu gerçekten hareketle grafiksel gösterim yöntemleri bulunmuş, bu yöntemleri kullanan yazılım araçları geliştirilmiştir.

Gösterim şekillerinin iyi bilinmemesi sonucu tasarımı yanlış anlaması, hatalı kodlamaya neden olabilir.

“Grafiksel gösterimlerin iyi öğrenilmesi ve anlaşılması gereklidir.”

Yapısal çözümleme ve tasarım :

Yapısal çözümleme ve tasarımda veri akış diyagramları ve durum geçiş diyagramları kullanılır.

UML :

Nesneye yönelik çözümleme ve tasarımın hem metinsel hem de grafiksel olarak yapılabilmesine yardımcı olan uluslar arası çevrelerce kabul edilmiş, standart ve yaygın bir tanımlama dilidir.

Akış diyagramları :

Çeşitli tasarım yöntemlerinde kullanılabilecek görsel anlatımları ve diyagramları ikiye ayırmak mümkündür.

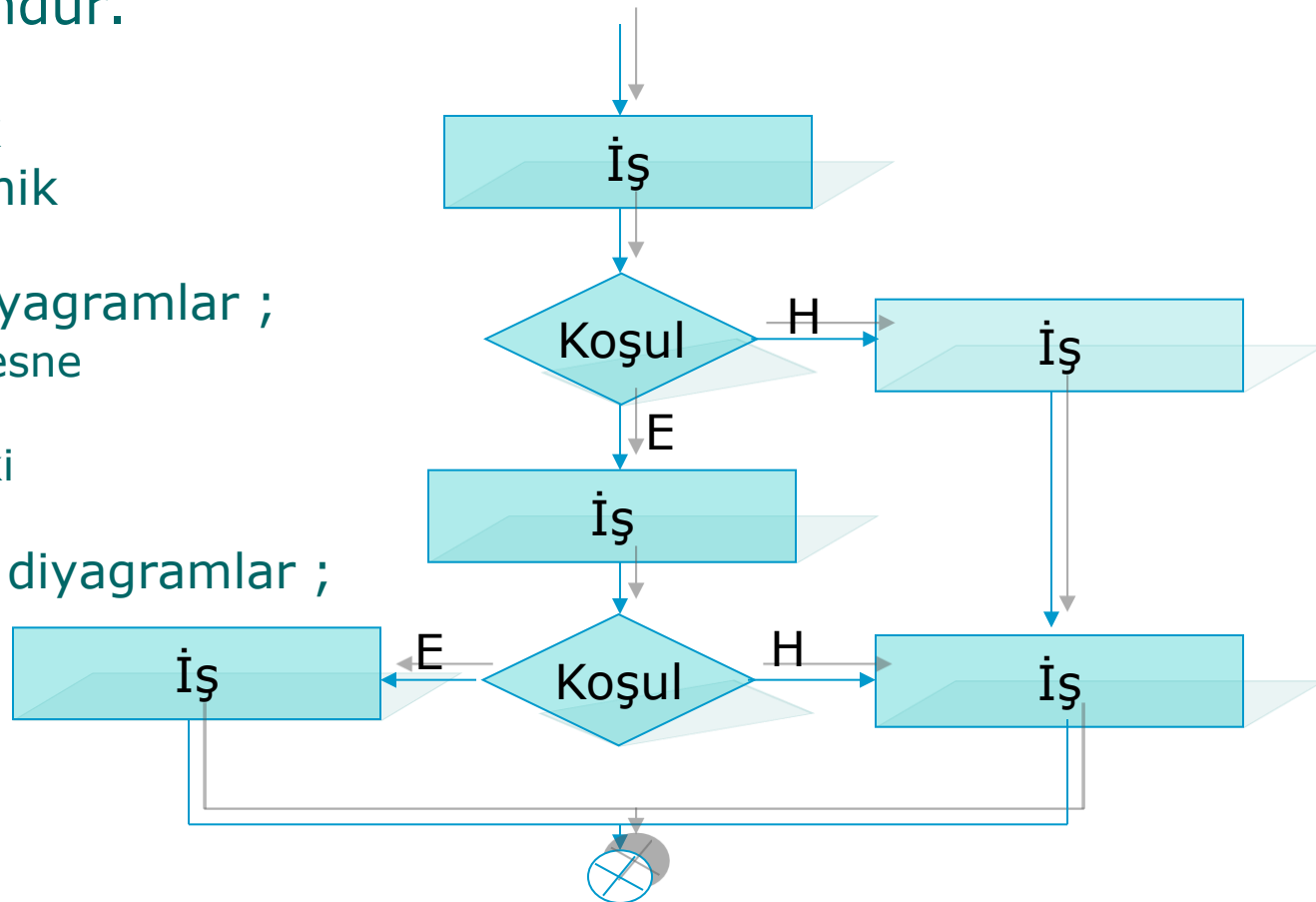
1. Statik
2. Dinamik

Statik akış diyagramları ;

Sınıf ve nesne
Bileşen
Varlık-iliski
Yapı ...

Dinamik akış diyagramları ;

Veri akış
Etkileşim
Durum
Akış ...



- <http://e-bergi.com/2008/Ekim/Cevik-Modelleme-ve-Cevik-Yazilim-Gelistirme>
- Sistem Analizi ve Tasarımı Prof.Dr. Oya Kalıpsız
- Yazılım Mühendisliği Dr.M.Erhan Sarıdoğan
- BT HABER dergisi, Sayı 259, 2000.
- DELPHI UNLEASHED, SAMS PUBLISHING, Charles Calvert, 1997.
- www.mehmetduran.com
- <http://jamshidhashimi.com/2010/08/23/agilecevik-modelleme-ve-cevik-yazilim-gelistirme>
- http://en.wikipedia.org/wiki/Agile_Modeling
- http://en.wikipedia.org/wiki/Agile_software_development
- <http://www.minepla.net/2008/10/agilecevik-yazylym-gelithirme>

